

Informe análisis: “*Proyecto Final*”

Estudiantes:

Harlin David Palacios Bermúdez

Sebastian Arango Sánchez

Juan Pablo Moncada

Docente Informática II:

Aníbal José Guerra Soler

Universidad De Antioquia

Noviembre 06 del 2025

Segundo semestre

Momento_1 - Actualizado:

Tema: Desembarco de normandía (Dia D)

Contexto:

Remontándonos a la época de hace más de 50 años atrás, bajo el acontecimiento de la Segunda Guerra Mundial. Este fue un acontecimiento de gran relevancia histórica referente a la capacidad vil del egoísmo humano, las ansias de poder, dominio y orgullo frente a las demás naciones existentes, las cuales hoy reconocemos como potencias mundiales. Esta guerra, como bien mencionamos, fue devastadora y atroz, por lo que provocó diferentes cambios, tanto social como económicamente, gracias al gran derroche humano para el desarrollo de esta misma, la cual acrecentaba cada vez más y más, bajo el subyugo del ideal Nazi.

Sin embargo, gracias al heroísmo de un honorable grupo de guerreros, estos sacrificaron sus vidas para llevar a cabo la liberación de las desafortunadas naciones afectadas entre las que tenemos un pequeño, pero peculiar lugar perteneciente a Francia, la región de Normandía, el principio del todo. En este espacio se desató una fugaz infiltración por parte de las fuerzas aliadas (Estados Unidos, Reino Unido y parte de Francia Libre). Entonces el 6 de Junio de 1944, se inició el proceso de bombardeo naval y marítimo contra las fuerzas armadas Alemanas que dominaban sobre dicho escenario francés, para posteriormente desatar una infiltración por el canal de la Mancha, que dejó como producto alrededor de más de 10.000 muertes que hoy en día representan su valentía, sacrificio y cooperación internacional por el fin del dominio del imperio Alemán en Francia, como así mismo, en el resto de Europa Occidental.

Sinopsis-Video juego:

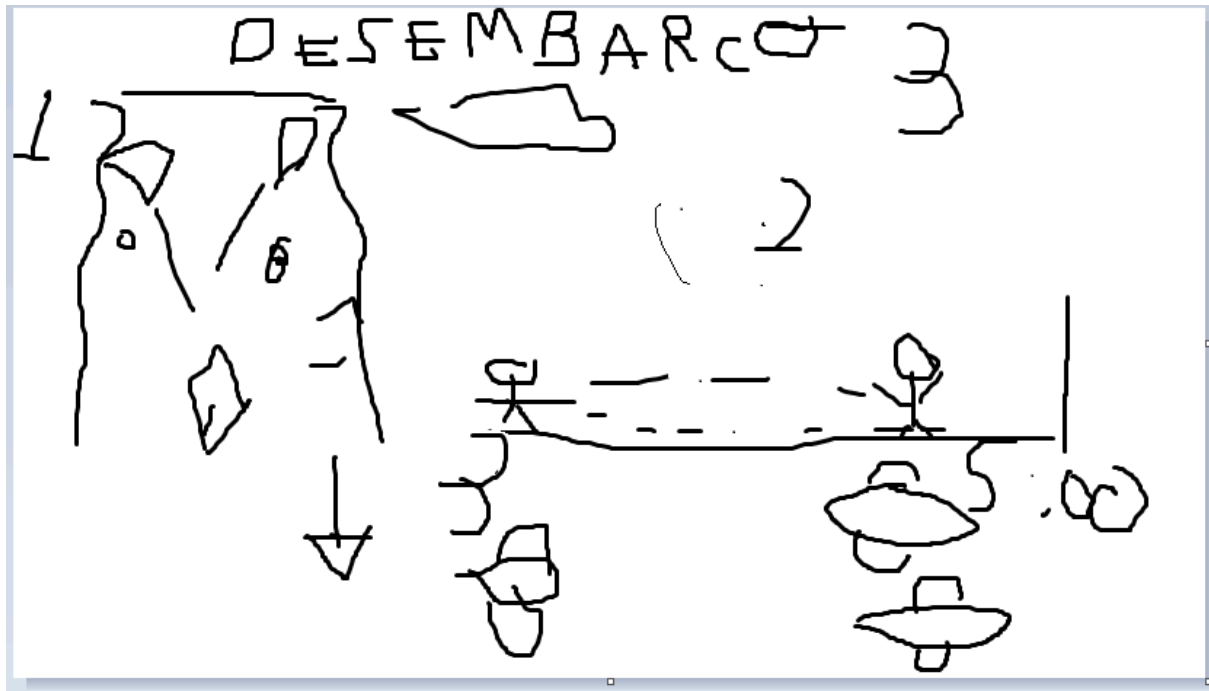
En el marco de una brutal guerra mundial, se encuentra una decadente Francia sin recursos para sobreponerse sobre el mandato Nazi impuesto por el dirigente Adolf Hitler, en diferentes espacios de este país. Actualmente nos encontramos en la región del Norte de Francia, Normandía, en camino al desarrollo de una infiltración y ataque de emboscada con el fin de debilitar las fuerzas Alemanas que azotan nuestro país Aliado, Francia. Bajo el nombre de Reino Unido y Estados Unidos, encaminaremos esta feroz senda hacia la liberación y el primer paso para el fin de esta Guerra Mundial, ¿Te sientes listo para esto cadete? ¿Dejarás la vida por tu país y defender la paz Mundial?

Cada ola, cada disparo y cada paso marcarán la diferencia entre la victoria y la derrota. El destino del mundo libre depende de tu valor.

Ármate de valor, contando con el apoyo de tus camaradas en el frente de guerra para bombardear las tropas enemigas de forma naval y aérea, para al final adentrarte al corazón de la Región, por tierra, para culminar la liberación del dominio Alemán.

¿Estás listo para desembarcar?

“Recuerda, hoy son nuestros estimados vecinos, en un futuro podría ser tu familia. Rendirse nunca será una opción cadete” - Att: Fuerzas Aliadas Unidas.



Acto-Nivel #1: Llegada Aérea



Vista: Lateral 2D con Scroll

Dinámica: Como primer nivel el jugador tendrá que disparar a barcos y aviones para defenderse, despejando el camino para la demás flotas aliadas, facilitándoles el paso y así, poder llegar sin dificultades de un posible ataque aéreo, o acuático a la costa.

Retos-Objetivos: El jugador tendrá como misiones:

- 1- Derribar todos los navíos y embarcaciones enemigas.
- 2- Adquirir todas la mayor Cantidad de Banderas Francesas Posibles, las cuales representan, un bono de apoyo para el Nivel Final.
- 3- No morir. (Obligatorio).

Físicas: Projectiles propios y de enemigos, Movimiento del Avión.

Ecuaciones: Parabólica para proyectiles a barcos, Movimiento Lineal Uniforme para Projectiles Aéreos y Provenientes de Barcos, Caída Libre para Movimiento del Avión.

Funcionamiento General: El jugador deberá desplazarse por todo el espacio de juego, disparando a los enemigos para derrotarlos y así poder superar el nivel, recolectando diferentes flags que delimiten que han recuperado parte del territorio, hasta poder eliminar a todos los rivales y terminar el nivel.

Acto-Nivel #2: Llegada en barco

Vista: Isométrica 2.5D

Dinámica: Se basa en llegar a la costa de Francia (Normandía) con un barco, solo esquivando rocas y navíos enemigos destruidos que se interponen en el camino, mientras se controlan las olas del mar enfurecido que hacen que el barco tenga un movimiento errático de sí mismo y de los obstáculos.

Retos-Objetivos: El jugador tendrá como misiones:

- 1- Esquivar todos los obstáculos que encuentre en el Camino.
- 2- Llevar a salvo a la tripulación a la orilla, evitando toda clase de pérdida de los cadetes tripulantes.
- 3 - Recibir el menor daño posible.

Físicas: Olas del mar que causan movimiento impredecible, Objetos en movimiento que pueden afectar la trayectoria del barco.

Funcionamiento General: El jugador deberá desplazarse por todo el espacio de juego, esquivando los diferentes objetos u obstáculos que se le presenten en el camino de forma que pueda llevar sanos y salvos a todos los cadetes hacia la costa de Normandía, por tanto, se deberán evitar las bajas aliadas en este proceso, con el fin de contar con la mayor cantidad de apoyo durante el último nivel.

Acto-Nivel #3: Supervivencia

Vista: Cámara cenital con Scroll.

Dinámica: Como nivel final el usuario tendrá que resistir un tiempo (*timer*) antes de que lleguen los refuerzos, entonces a este se acercan enormes oleadas de enemigos, los cuales este se defenderá con un arma. El jugador contará con un número limitado de vidas para realizar este nivel, además, podrá saltar para poder movilizarse en el entorno. Si se acaban las vidas el jugador morirá y empezará desde cero el nivel.

Retos-Objetivos: El jugador tendrá como misiones:

- 1- Sobrevivir el tiempo estipulado.
- 2- No morir. (Obligatorio).

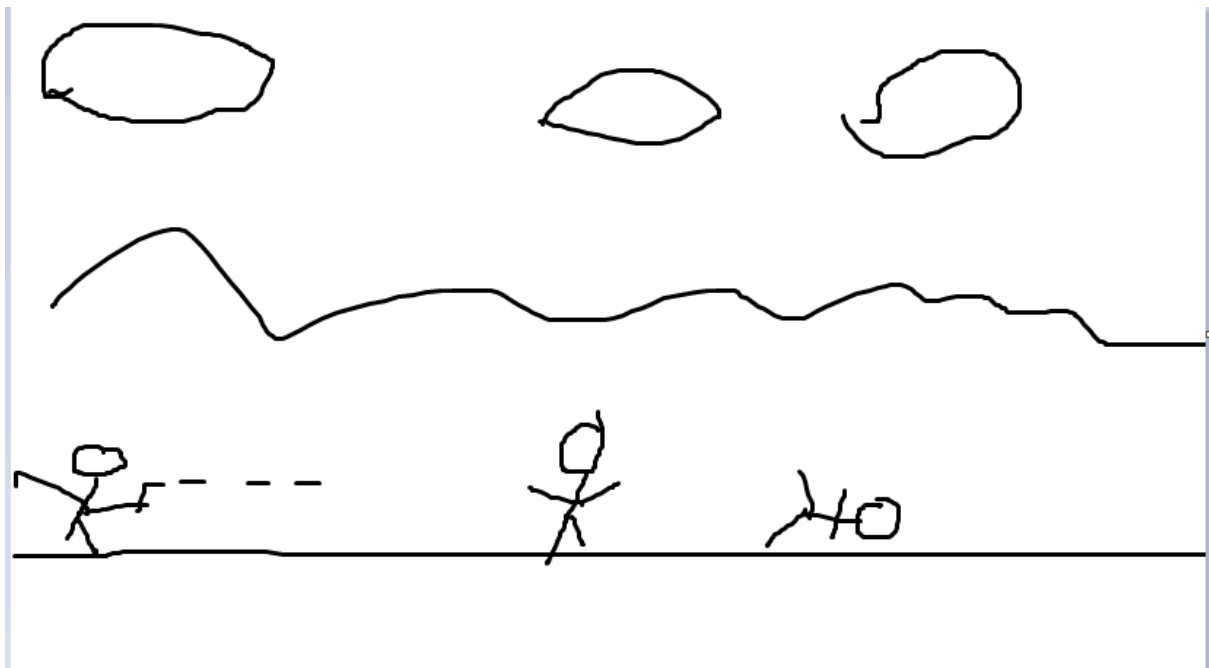
3- Derrotar a todos los enemigos.

4- Liberar el territorio terrestre Francés del Dominio Alemán.

Físicas: Proyectiles propios y de enemigos, además de físicas al saltar.

Ecuaciones: Movimiento Parabólico para actividades como saltar y lanzar Armas (Proyectiles).

Funcionamiento General: El jugador deberá desplazarse por todo el espacio de juego, disparando a los enemigos para derrotarlos y así ahuyentar a todas las tropas Alemanas del territorio Francés, estando alertas ante todos los ataques posibles que puede recibir y resistir a las diferentes oleadas (Sin perder todas sus oportunidades de vida), con el fin de terminar el juego.



Componente Investigativo:

Este se plasmará inicialmente como una alternativa para la mayoría de los niveles, lo cual permitirá que las acciones que desarrollen los enemigos puedan representar un pequeño cambio en sus acciones, ya sea desplazamiento o decisiones a tomar, para contrarrestar los métodos que utilice el jugador para derribarlos. Así mismo también nos es posible plantear técnicas de un compañero autónomo que sea de gran ayuda ante fuertes oleajes de enemigos en el nivel final, como un tipo de agente de apoyo (Fase en Desarrollo).

Referencias:

<https://github.com/Przemekkkth/top-down-game-sfml?tab=readme-ov-file>

<https://www.youtube.com/watch?v=iBdX2wcM8ag>

<https://github.com/johnBuffer/ZombieV?tab=readme-ov-file>

<https://www.youtube.com/watch?v=pj3m3Fu3i5A>

Imagenes (IA generativa):

Momento II:

Acto-Nivel #1: Llegada Aérea

Descripción Vista:

Para este nivel, como hemos descrito anteriormente, se desarrollará bajo una vista **lateral 2D con Scroll**.

Bajo este contexto, el entorno en el cual se desarrollará la batalla aérea se dará en torno al océano, representando un recorrido hacia las cercanías de la costa del norte Francés Normandía. El jugador podrá evidenciar su desplazamiento en el nivel, de forma tal, que todo el entorno se desplaza menos él, ya que este se encontrará moviéndose en una determinada posición x (Eje y), hacia arriba y hacia abajo.

Interacciones Personajes: El jugador, quien controlara un navío personalizado, disparará a los navíos y embarcaciones enemigas, tratando de esquivar los proyectiles que le lancen, y así mismo, este haciendo todo lo posible por derrotar a los rivales, de forma que, permita debilitar las fuerzas del ejército Alemán y de paso al despliegue de sus compañeros por vía marítima. Su misión será sobrevivir el Nivel, evitando los proyectiles enemigos para no morir y destruir la mayor cantidad de enemigos como le sea posible.

Físicas: Caída Libre para modelar el desplazamiento del avión, hacia arriba y hacia abajo. Movimiento Rectilíneo para los proyectiles enemigos y Movimiento Parabólico para Proyectiles del Avión del Jugador hacia los barcos.

Agente Inteligente:

Este se implementará en este nivel, con el propósito de realizar un cambio en las dinámicas de Ataque de las Embarcaciones y Navíos enemigos, haciendo que el usuario tenga que afrontar nueva clase de disparos que afecten su integridad, por tanto, deberá rápidamente optar a buscar nuevos mecanismos para defenderse ante las fuerzas enemigas, y el drástico comportamiento por el que estos optarán en un determinado punto del nivel. En donde, las nuevas dinámicas de ataque modelarán una acción en conjunta por varios de los enemigos aéreos, en la cual estos se ubicaran en distintas posiciones respecto al Eje y, disparando de forma continua, en un intervalo específico de tiempo, en el cual, pasado dicho momento el navío enemigo irá alternando sus posiciones hasta destruirse por sí mismo.



Acto-Nivel #2: Llegada en barco

Descripción vista: Isométrica 2.5 D

Interacciones del nivel: Como se mencionó en el momento, este nivel se basará en un medio acuático manejando un barco, el cual tendrá que esquivar las rocas, barcos enemigos hundidos, también contará con efectos de colisión, el cual aunque no tenga ninguna afectación en el nivel, se necesita para poder verlo más realista. El jugador tendrá la misión de resguardar lo mayor posible el navío para que llegue de la mejor manera posible a la costa y poder pasar al siguiente nivel.



Continuamos con el mismo contexto que tuvimos en el momento, el cual era llegar con el barco a la costa de Francia, para poder tomarla del enemigo, además como se mencionó la vista será Isométrica 2.5 D.

La jugabilidad del nivel solo será esquivar, se puede mover por todo el plano, o sea, izquierda, derecha, arriba y abajo.

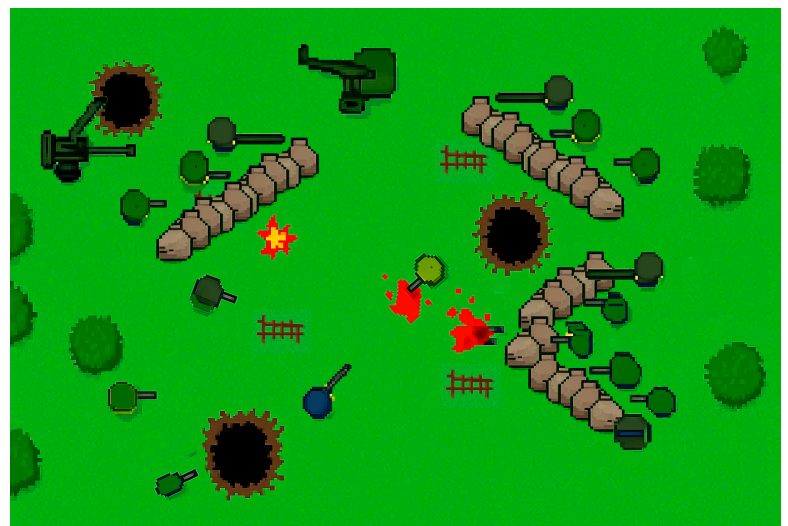
Acto-Nivel #3: Supervivencia

Vista: Cenital con scroll.

Escenario: El escenario transcurre en la playa de Omaha Beach, cerca de Vierville-sur-Mer, Normandía (Francia)

Dinámica:

Como nivel final el usuario tendrá que resistir un tiempo (**timer**) antes de que lleguen los refuerzos, entonces a este se acercan enormes oleadas de enemigos, los cuales este se defenderá con un arma. El jugador contará con un número limitado de vidas para realizar este nivel, además, podrá saltar y agacharse para poder movilizarse en el entorno. Si se acaban las vidas el jugador morirá y empezará desde cero el nivel.



Agente inteligente:

El agente inteligente funcionará haciendo que los enemigos que tiene que combatir el jugador vayan aumentando en número conforme pasa el tiempo y además que cambien su estrategia. Al principio se dirigirán hacia el jugador buscando la ruta más corta sin importar nada. Pasado un tiempo empezaran a agruparse y esconderse para que el jugador no pueda dispararles y que cuando estén a cierta distancia salgan en conjunto a por el jugador, así formando grupos de enemigos que emboscan el jugador y lo acorralan conforme avance el tiempo.

Retos-Objetivos: El jugador tendrá como misiones:

- 1- Sobrevivir el tiempo estipulado.
- 2- No morir. (Obligatorio).
- 3- Derrotar a todos los enemigos.
- 4- Liberar el territorio terrestre Francés del Dominio Alemán.

Funcionamiento General: El jugador deberá desplazarse por todo el espacio de juego, saltando, agachándose y escondiéndose detrás de los objetos del escenario y disparando a los enemigos para derrotarlos y así acabar con todas las tropas Alemanas del territorio Francés, estando alertas ante todos los ataques posibles que puede recibir y resistir a las diferentes oleadas (Sin perder todas sus oportunidades de vida), con el fin de terminar el juego.

Agentes inteligentes

1. Qué Ve el Agente:

Posición del jugador: Dónde está y hacia dónde se mueve

Obstáculos: Rocas, barcos destruidos.

Amenazas: Balas y proyectiles cercanos

Objetivos: Detener al jugador

2. Cómo Decide:

Si ven al jugador → disparan

Si el jugador se acerca mucho → se alejan

Si el jugador usa mucho una estrategia → la contrarrestan

3. Qué Hace:

Moverse: Hacia el jugador o lejos de él

Disparar: Con diferentes tipos de balas

Esquivar: Balas y obstáculos.

4. Cómo Aprende:

Recuerda lo que hizo el jugador recientemente.

Cambia su estrategia si algo no funciona.

Se adapta a cómo juega la persona.

Imágenes Comportamiento con Agente Inteligente



Link al Diagrama de clases:

https://lucid.app/lucidchart/99e718fb-8c18-48ee-93c4-90c02677c70c/edit?invitationId=inv_f3fa39e1-09a3-47db-a5ab-8bdeeca6cc15&page=0_0#

Sprites



INFORME_FINAL:

Informe de responsabilidades:

Harlin Palacios:

- Diagrama de clases
- Desarrollo del nivel 1 (+agente inteligente, +física, scroll)

Juan Paulo Moncada:

- Diagrama de clases
- Desarrollo de nivel 3 (+agente inteligente, nivel con oleadas)
- Implementación de sprites en todos los niveles

Sebastian Arango:

- Desarrollo del menú
- Desarrollo del nivel 2 (+física, vista 2.5D, tiempo)
- Implementación de sonido para todos los niveles
- Edición de videos.

1- Introducción:

Nuestro proyecto final es un videojuego basado en el Desembarco de Normandía, donde recreamos tres momentos clave de esta operación militar. El juego tiene tres niveles diferentes: en el primero manejamos un avión que dispara a barcos enemigos, en el segundo controlamos un barco que debe esquivar obstáculos en el mar, y en el tercero sobrevivimos a oleadas de enemigos en la playa. Encontramos varios desafíos, como hacer que los enemigos se comporten de forma inteligente y unificar los diferentes sistemas del juego, logramos avanzar en la implementación de cada nivel con los conocimientos que hemos adquirido en el semestre actual.

2- Contexto del proyecto, motivación y público objetivo:

Contexto del proyecto:

El proyecto se sitúa en el Desembarco de Normandía del 6 de junio de 1944, uno de los momentos más decisivos de la Segunda Guerra Mundial. La operación militar, conocida como "Día D", involucro la llegada de Estados Unidos a la guerra, en una invasión anfibia masiva para liberar a Francia del dominio nazi. Este acontecimiento marcó el inicio de la liberación de Europa a manos de Alemania, representando un punto de inflexión crítico que eventualmente condujo a la derrota de la alemania nazi en 1945.

Motivación:

El juego busca transmitir valores como valentía, trabajo en equipo y resistencia ante adversidades, el proyecto representa un desafío técnico al implementar tres paradigmas visuales diferentes (lateral 2D, isométrico 2.5D y cenital con scroll) con sistemas de inteligencia artificial que emulan tácticas militares reales como agrupamiento y adaptación al comportamiento del jugador.

Público Objetivo:

El software está diseñado para jugadores casuales de 10 años para adelante, que disfrutan de juegos de acción arcade. El público incluye aficionados a juegos de guerra que valoran y gamers indie interesados en mecánicas de IA adaptativa y múltiples perspectivas visuales. El juego también atrae a desarrolladores que desean estudiar implementaciones de diferentes sistemas de colisión, física de proyectiles y arquitecturas de juego en Qt/C++. La clasificación recomendada es +10 por temática bélica, aunque la violencia es matizada, sin representaciones gráficas explícitas, priorizando el aspecto estratégico y táctico sobre el contenido sensible.

3- Planteamiento del problema:

Cómo diseñar una experiencia interactiva que preserve el rigor histórico del Desembarco de Normandía mientras implementa mecánicas de juego atractivas que generen conexión con los eventos, manteniendo coherencia a través de tres perspectivas operativas distintas (aérea, naval y terrestre).

Problemas técnicos al programar:

- Unificar tres vistas diferentes (lateral 2D, isométrica 2.5D, cenital) en una experiencia coherente
- Implementar sistemas de IA adaptativa que respondan al comportamiento del jugador
- Mantener estable a través de mecánicas diversas (disparos, movimiento, colisiones)
- Manejo coherente entre clases abstractas como base de todo
- Desarrollo de una estructura adecuada de manejo de pertenencia de datos y responsabilidades de las clases involucradas en cada nivel

4- Necesidad que cubre el software y justificación del desarrollo:

El software cubre la necesidad de ofrecer una experiencia interactiva que permita comprender, de forma didáctica y entretenida, un acontecimiento histórico relevante como el Desembarco de Normandía. Además, sirve como una herramienta académica para aplicar y reforzar conocimientos de programación orientada a objetos, diseño de videojuegos, manejo de gráficos, físicas básicas e inteligencia artificial. El desarrollo del proyecto se justifica como una oportunidad para integrar teoría y práctica, fomentando el trabajo en equipo, la planificación y el uso de herramientas profesionales como Qt Creator y GitHub, para rememorar un acontecimiento histórico de nuestra historia humana, en la cual se relata el producto del cinismo y egoísmo humano, lo cual nos llevo a desencadenar graves catástrofes, mas sin embargo, refuerza valores como el aprecio al reconocimiento de la vida humana misma y a contemplar la unión de nuestras diferencias por un bien común.

5- Definición general y objetivos:

El proyecto esta inspirado en el Desembarco de Normandía (Día D) de la Segunda Guerra Mundial. Su objetivo principal es crear una experiencia interactiva con mecánicas de juego

atractivas, ofreciendo tres niveles distintos: un nivel aéreo con vista lateral y scroll, un nivel naval en perspectiva isométrica y un nivel de supervivencia terrestre con vista cenital.

Entre los objetivos específicos se incluyen implementar sistemas de movimiento y colisiones, diseñar enemigos con comportamientos básicos de inteligencia artificial, y garantizar una jugabilidad fluida y desafiante.

6- Alcance funcional, metas y requisitos principales:

El videojuego abarca la recreación interactiva de tres momentos principales del Desembarco de Normandía mediante tres niveles jugables independientes, cada uno con una perspectiva y dinámica distinta: un nivel aéreo con vista lateral 2D, un nivel naval con vista isométrica 2.5D y un nivel terrestre de supervivencia con vista cenital y scroll.

El alcance del proyecto se limita a una experiencia de tipo arcade, enfocada en la jugabilidad y el aprendizaje técnico, sin pretender una simulación histórica exacta de los acontecimientos reales.

El proyecto tiene como meta principal el desarrollo de un videojuego funcional utilizando programación orientada a objetos en C++ sobre el framework Qt Creator, integrando sistemas de movimiento, disparos, colisiones y manejo de estados.

Se busca implementar y unificar tres paradigmas visuales diferentes, manteniendo coherencia entre niveles y una transición lógica entre ellos.

Otra meta importante es el diseño de enemigos con comportamientos básicos de inteligencia artificial, capaces de reaccionar ante las acciones del jugador y adaptarse progresivamente durante el desarrollo de cada nivel.

Entre los requisitos funcionales principales se incluyen el control del jugador mediante teclado y mouse, sistemas de vida y daño, gestión de proyectiles, detección de colisiones, condiciones de victoria y derrota, y la visualización de información relevante a través de un HUD.

Como requisitos no funcionales se establece un rendimiento estable cercano a 60 FPS, una interfaz clara e intuitiva para jugadores casuales, un código modular y organizado, y el uso eficiente de recursos gráficos y de audio.

7- Especificación de requerimientos mínimos:

- **Computador con las siguientes:**
 - **Procesador : Intel core i3 10th generación**
 - **Grafica : Graficos integrados**
 - **Memoria RAM : 4GB**
 - **Memoria : 200MB**

8- Requisitos funcionales, requisitos no funcionales, restricciones, dependencias y usuarios destinatarios:

Movimiento lateral del barco con las teclas izquierda y derecha, sprint con la tecla de arriba, sistema de disparo de torpedos con espacio, generación progresiva de obstáculos, detección de colisiones, sistema de vidas y munición, dificultad incremental cada 5 segundos, objetivo de supervivencia de 20 segundos, efectos de sonido para disparos y explosiones, pantallas de victoria/derrota.

Rendimiento estable de 60 FPS, controles intuitivos, código modular. DEPENDENCIAS: Qt Core, GUI, Widgets, Multimedia.

USUARIOS: Jugadores casuales.

```
FLUJO NIVEL BARCO:  
[INICIO] → [Configuración Inicial] → [Bucle 60 FPS]  
    └─ Entrada: Movimiento + Disparos  
    └─ Actualizar: Obstáculos + Torpedos + Colisiones  
    └─ Dificultad: Aumento progresivo  
    └─ Render: Escena isométrica  
      ↓  
[Verificar] → [VICTORIA: 20s] o [GAME OVER: 0 vidas]
```

Aquí tienes la misma sección (8) pero para el Nivel 3, con el mismo estilo del ejemplo del nivel 2 (texto + mini diagrama de flujo).

Nivel 1:

Para el desarrollo de este respectivo nivel, se dio en práctica una aplicación de una dinámica con vista lateral 2D, en la cual gozando de un espacio aéreo, se llevan a cabo confrontaciones entre el bando de las tropas aliadas y el bando alemán, acrecentando una disputa aérea, en la cual el jugador (Guerrero de la tropa Aliada), debe esquivar, disparar y atravesar todo el escenario, infligiendo el mayor daño posible para acabar con las tropas enemigas y sobreviviendo a las dificultades diversas del nivel que se puedan presentar como desplazamientos anormales, gran cantidad de misiles amenazantes, vasta cantidad de enemigos, imposibilidades de movilidad, entre otras problemáticas respectivas al caso. Entonces, con todo esto anterior mencionado, el nivel satisface las siguientes necesidades:

- Dificultad de Jugabilidad: Representando no ser de forma excesiva, para el Jugador hay inmensidad de trabas que lo llevaran a tratar de realizar las mejores decisiones para su supervivencia, tomando caminos óptimos, destruyendo aviones en momentos oportunos, ya que cada decisión desempeña o encadena una posterior acción a ejecutar, ya sea luego de atacar esquivar, o viceversa.
- Fluidez: Un correcto desempeño de cada movimiento para el cual el jugador logre de la mejor manera esquivar, y disparar a los enemigos.
- Sorpresividad: en cada frame el futuro del jugador es incierto, debido a la generación aleatoria de enemigos en diferentes posiciones generando rutas fáciles o muy difíciles de esquivar, por esto, el jugador debe estar atento a que estrategia utilizar y priorizar las rutas por las cuales menor daño deberá asumir.
- Cambios Comportamentales: Según las diferentes etapas de juego, es posible encontrar cambios en el desplazamiento del enemigo y por tanto, mayor agilidad y concentración, sin

embargo, se optimiza la aparición de enemigos para que sea posible sobrepasar la etapa, desde que la concentración prime, porque cada descuido puede ser letal.

- **Visualización Datos:** En cada etapa, y transcurso del juego, se favorece al usuario visualizar sus estadísticas de juego, para saber que tanto hace falta para finalizar el nivel, el progreso de sus logros, y sobre todo tener una idea o previo aviso de la cantidad de vida que posee y cuanto daño mas puede abarcar, por tanto, esto favorece al jugador en su toma de decisiones para desempeñarse de mejor forma en cada instancia.
- **Pérdidas Significativas:** Representadas con las colisiones, estas afectarán la integridad del Avión del jugador de forma que este poco a poco irá perdiendo su firmeza y estabilidad hasta el punto de en cada choque colapsar, así mismo como sucederá con los enemigos mismos.

Nivel 3:

Requisitos funcionales (qué hace el nivel):

- **Movimiento del jugador con teclado (WASD / flechas) mediante scroll del fondo/cámara.**
- **Apuntado del jugador con el mouse (la dirección del cadete cambia según el cursor).**
- **Sistema de disparo con click izquierdo: crea proyectiles en la dirección actual del jugador.**
- **Sistema de munición: cada disparo consume 1 bala; si no hay balas, no dispara.**
- **Recarga manual con tecla R:**
 - **Muestra barra de recarga en HUD.**
 - **Al terminar, restaura munición al máximo.**
 - **Click izquierdo durante recarga cancela la recarga.**
- **IA por oleadas (Agentes / OleadaCadetes):**
 - **Spawn de enemigos alrededor del jugador por ronda.**
 - **Modos de comportamiento: AtaqueDirecto, Campamento, Rotación.**
 - **Enemigos se mueven hacia el jugador o hacia focos (campamento/retirada).**
 - **Enemigos disparan con cooldown grupal (ráfagas limitadas por tick).**
- **Colisiones:**
 - **Jugador vs obstáculos: impide atravesar (se revierte movimiento de cámara/fondo).**
 - **Enemigos vs obstáculos: reacción de colisión (corrige movimiento).**
 - **Enemigos vs enemigos: evita solaparse (separación).**
- **Sistema de vida:**
 - **Jugador y enemigos reciben daño por proyectiles (según si el proyectil es de jugador/enemigo).**
 - **Si el jugador muere → Game Over.**
- **Sistema de rondas:**
 - **3 rondas con configuraciones distintas (grupos/slots/modos).**
 - **Al eliminar todos los enemigos de una ronda, avanza a la siguiente.**
 - **Al completar todas → Victoria.**

- **HUD:**
 - Barra de vida, ronda actual, contador de enemigos, munición, barra de recarga.
- **Pantallas de fin:**
 - Overlay de Victoria / Game Over, con stats (rondas, enemigos, vida final), botones reintentar y volver al menú.

Requisitos no funcionales (calidad / rendimiento):

- Rendimiento estable cercano a 60 FPS (timer ~16ms).
- Sprites en pixel-art nítido (sin suavizado al escalar/rotar).
- Código modular por capas: Nivel (orquestador) + Agentes + Entidades + Proyectiles + Obstáculos.
- Escalabilidad: soporta múltiples enemigos y rondas sin re-estructurar el loop.

Restricciones (límites del diseño actual):

- El jugador está “fijo” en el centro (0,0) y el mundo se mueve (cámara/fondo).
- El tamaño del sprite depende de `boundingRect()` y del radio: si el sprite se ve gigante o se recorta, hay que ajustar el rect real + rotación.
- La IA y los disparos dependen del timer del nivel: si el timer se detiene, todo queda congelado.

Dependencias:

- Qt Core, GUI, Widgets, Multimedia
- QGraphicsScene / QGraphicsView / QGraphicsItem
- Recursos `.qrc` para sprites (cadetes, obstáculos, fondo si lo pasas a qrc)
- (Opcional) QSoundEffect para ambiente/disparos (si lo activas también en nivel 3)

Usuarios destinatarios:

- Jugadores casuales (arcade/top-down), que buscan rondas cortas, disparo simple y feedback claro en HUD.

Flujo del nivel 3: Cadetes (diagrama)


```

FLUJO NIVEL 3: INFANTERÍA (Cadetes)

[INICIO] -> [Setup: escena + fondo + chunks + obstáculos + jugador + HUD + audio] -> [Bucle ~60 FPS]
|
+--> Entrada: WASD (scroll/cámara) + Mouse (apuntar) + Click (disparo) + R (recarga)
|
+--> Actualizar:
    - Movimiento + colisión jugador/obstáculos (revert cámara)
    - IA agentes (oleadas: ataque/campamento/rotación)
    - Proyectiles (avanzar + colisiones + limpiar)
    - Enemigos muertos (desactivar/remover)
    - HUD (vida, ronda, enemigos, balas, recarga)
|
+--> Verificar:
    - [GAME OVER: jugador sin vida]
    - [VICTORIA: rondas completas]

```

9- Metodología y planificación:

Se utilizó el lenguaje de programación C++ utilizando Programación orientada a objetos en el framework Qt Creator y la combinación de Git y github para el desarrollo paralelo.

- Estrategia de responsabilidades por persona
 - Coordinador general Juan Paulo, se encargó de manejar el repositorio y vigilar la correcta implementación de cada parte del desarrollo.
 - El desarrollo se distribuye en etapas base -> lógica -> visual -> final
Cada persona trabaja en un nivel específico dependiendo de la etapa, así cada uno trabaja de manera independiente. Después de terminar cada etapa se juntan los resultados para decidir quien trabaja en qué nivel para la siguiente etapa.
 - Desarrollo paralelo con git:
Se utilizó una rama dev como rama de trabajo.
En cada etapa había una persona encargada del desarrollo de esta en un nivel en específico
Todos los merge a la rama dev se hicieron desde github con revisión y manejo de errores de parte de Juan Paulo.
 - A la hora de terminar cada etapa de cada nivel se hace merge desde github a la rama dev para juntar el desarrollo individual de cada persona

11- Diseño y arquitectura del sistema:

El videojuego está diseñado bajo una estructura jerárquica, la cual se basa en el uso adecuado de las herramientas proporcionadas por el entorno de desarrollo Qt Creator. Como base principal del sistema se utiliza la clase QMainWindow, que funciona como el contenedor general donde se integra toda la aplicación.

Dentro de esta ventana principal se emplean los componentes QGraphics, los cuales permiten asociar de manera individual cada objeto del juego con una representación visual. De esta forma, elementos como barcos, aviones, cadetes, obstáculos y otros objetos del entorno pueden ser mostrados correctamente dentro del nivel, facilitando su identificación y manejo visual.

Una vez establecida la parte gráfica, se organiza la lógica del juego siguiendo un orden claro. En el nivel superior se encuentra el Nivel, el cual se encarga de controlar el desarrollo general de la partida. Debajo de este se agrupan todos los elementos que pertenecen al nivel, incluyendo los entes y los proyectiles, los cuales dependen directamente del nivel para su funcionamiento, interacción y actualización durante el juego.

Esta forma de organización permite mantener el sistema ordenado, facilita el control de los objetos y hace posible que el videojuego pueda ampliarse o modificarse de manera sencilla en futuras etapas del desarrollo.

12- Explicación jerárquica de los componentes:

Profundizando en la estructura de la capa lógica, se establece que todos los elementos que intervienen en la simulación pertenecen a un nivel, el cual actúa como contenedor principal y responsable de su gestión. Dentro de este nivel se definen distintos entes, cada uno con un comportamiento y propósito específico.

En el desarrollo se contemplan entes con personalidades diferenciadas, tales como barcos, aviones y cadetes, los cuales comparten una base común, pero poseen características propias de movilidad, interacción y respuesta. Estas diferencias permiten que cada ente se adapte de manera adecuada a los distintos escenarios del juego, ejecutando acciones particulares como desplazamiento direccional, disparos, detección de colisiones y reacciones ante eventos del entorno.

A su vez, dentro de esta jerarquía se encuentran los proyectiles, los cuales se consideran entes especializados derivados de un comportamiento común. Estos se subdividen principalmente en balas y misiles, cada uno con propiedades específicas como velocidad, trayectoria y tipo de impacto. Al igual que los entes principales, los proyectiles cuentan con su propio sistema de desplazamiento y mecanismos de interacción con otros entes, permitiendo la manifestación de choques, efectos secundarios y condiciones de resolución, como la destrucción de un objetivo o la activación de eventos lógicos.

Esta organización jerárquica facilita la escalabilidad del sistema, el mantenimiento del código y la reutilización de comportamientos comunes, permitiendo incorporar nuevos entes o proyectiles sin afectar la estructura general del nivel.

13- Diagrama general de clases, flujo y breves descripciones individuales:

https://lucid.app/lucidchart/99dde0f9-8ec6-4db7-b6d1-2cb829a0e41f/edit?page=0_0&invitationId=inv_7670598d-e095-44be-9c2b-db078651010c#

14- Dependencias externas y bibliotecas utilizadas:

- Qt Core: temporizadores, eventos, señales/slots y lógica base.
- Qt GUI + Qt Widgets: renderizado gráfico, manejo de entrada, HUD e interfaz.
- Qt Graphics View Framework: escena, vista y entidades del juego.
- Qt Multimedia: efectos de sonido y audio ambiente.

- Qt Resource System (.qrc): gestión de sprites y assets embebidos.
- STL de C++: contenedores, algoritmos y utilidades estándar.

15- Desarrollo e implementación:

El desarrollo del videojuego se realizó de forma incremental, comenzando por la estructura base del proyecto y continuando con la implementación de la lógica, los sistemas gráficos y las mecánicas de juego. Cada nivel fue desarrollado de manera independiente, permitiendo pruebas constantes y corrección de errores. Posteriormente, se integraron los sistemas comunes como colisiones, HUD, sonido y gestión de estados, logrando una experiencia de juego coherente y funcional.

16- Breve descripción de la lógica interna, principales algoritmos y decisiones técnicas, sin detallar todo el código fuente:

LÓGICA NIVEL 2:

El nivel 2 es isométrico y aplica una proyección matemática para crear una vista 2.5D, con movimiento lateral del jugador y scrolling automático de obstáculos. Los algoritmos principales incluyen un sistema de colisiones dual. También se desarrolló una clase Vector2D para operaciones matemáticas y un sistema de dificultad progresiva que ajusta la frecuencia y cantidad de obstáculos según el tiempo.

```
* - El barco solo puede moverse en el eje Y (izquierda/derecha)
* - El eje X está reservado para el scrolling automático
*
* Convención de ejes:
*   x (profundidad)
*   ^
*  /
* /
* -----> y (ancho)
*/
```

Las decisiones técnicas clave fueron la separación clara entre la lógica del juego en 2D y el renderizado, y un game loop fijo a 60 FPS. Se implementó un sistema de invulnerabilidad con parpadeo para evitar daños múltiples, además cuenta que los proyectiles no son disparados con un cooldown para que el jugador no tenga tanta ventaja. La arquitectura es modular, con clases separadas para cada nivel y componentes reutilizables como la gestión de colisiones y la proyección isométrica. Este diseño permite un flujo de juego estructurado que procesa input, actualiza estados, verifica colisiones y renderiza, manteniendo un código organizado y mantenible.

17-Procedimientos de prueba:

Las pruebas se realizaron de manera manual durante el desarrollo de cada nivel, verificando el correcto funcionamiento del movimiento, disparos, colisiones, sistemas de vida y transiciones entre estados. Se ejecutaron pruebas repetidas tras cada cambio importante en el código para asegurar la estabilidad del sistema y evitar errores críticos que afectan la jugabilidad.

18- Métodos de verificación, casos de prueba, resultados obtenidos y revisión de calidad.

Se verificó el correcto cumplimiento de los requisitos funcionales mediante casos de prueba como: eliminación de enemigos, detección de colisiones, pérdida de vidas, finalización de niveles y respuesta del HUD. Los resultados obtenidos fueron satisfactorios, ya que el sistema respondió de forma consistente en la mayoría de escenarios. La revisión de calidad se centró en la fluidez del juego, claridad visual y ausencia de fallos graves durante la ejecución. Ya que ante diferentes efectos de prueba todas estas condiciones que se mejoraron, cortaban la ejecución de las dinámicas de juego o peor aún afectaban los comportamientos esperados, siendo así, que esto enfatizó sumamente el cuidado de la visualización con el fin de hacer preferencial para el jugador su comodidad en el programa, disfrutando de una gran experiencia en torno a las acciones esperadas a realizar y el desarrollo íntegro de cada elemento que compone los niveles sin que se denoten aparatosos bugs que entorpezcan la idealización del juego.

19- Guía de instalación y uso:

El usuario debe instalar Qt Creator, abrir el proyecto proporcionado, compilarlo y ejecutarlo desde el entorno de desarrollo. Una vez iniciado el juego, se presenta un menú principal desde el cual se pueden seleccionar los niveles disponibles. El control del jugador se realiza mediante teclado y mouse, siguiendo las instrucciones implícitas del juego.

20- Instrucciones para instalar y ejecutar el software:

Paso 1: Instalar Qt. Descarga Qt desde **qt.io/download** e instala Qt Creator con el kit de desarrollo completo. Asegúrate de incluir Qt Multimedia en la instalación.

Paso 2: Obtener el código. Clona o descarga el repositorio del proyecto y extrae todos los archivos manteniendo la estructura de carpetas intacta: **Desafio_final/** debe contener **core/**, **entities/**, **levels/**, **utilities/** y **assets/**.

Paso 3: Configurar recursos. Verifica que la carpeta **assets/audio** contenga todos los sonidos de los respectivos niveles. Abre **assets/resources.qrc** en Qt Creator y confirma que ambos archivos de audio estén registrados bajo el prefijo **/audio**.

Paso 4: Abrir el proyecto. Lanza Qt Creator, selecciona "**Abrir Proyecto**" y navega hasta **Desafio_final.pro**. Selecciona el kit de compilación apropiado (Desktop Qt 5.15.2 MinGW o similar) y haz clic en "Configure Project".

Paso 5: Compilar. Presiona **Ctrl+B** (Windows/Linux) o **Cmd+B** (macOS) para compilar. El proceso generará el ejecutable en la carpeta **build-Desafio_final-Desktop-Debug/** o similar. Si hay errores de rutas de audio, verifica que **resources.qrc** esté correctamente configurado.

Paso 6: Ejecutar. Presiona **Ctrl+R** o el botón verde en Qt Creator. El juego iniciará mostrando el menú principal con opciones para seleccionar niveles.

Paso 7: Disfruta

21- Requisitos técnicos y recomendaciones.

Se recomienda ejecutar el juego en un equipo que cumpla o supere los requerimientos mínimos para garantizar una experiencia fluida. Es aconsejable cerrar otras aplicaciones durante la ejecución del juego y utilizar una resolución estándar para evitar problemas de escalado visual. También se recomienda contar con dispositivos de entrada funcionales como teclado y mouse.

22-Resultados y discusión

El proyecto logró implementar satisfactoriamente los tres niveles planteados, cada uno con mecánicas distintas y coherentes con su contexto. Se implementó un correcto uso de la programación orientada a objetos, de forma tal que se llegó al logro de cada una de las ideas plasmadas para el desarrollo y una integración adecuada de gráficos, físicas e inteligencia artificial. Aunque existen aspectos por mejorar, los resultados obtenidos cumplen con los objetivos académicos propuestos.

23- Funcionamiento logrado, limitaciones actuales, dificultades enfrentadas y mejoras planeadas.

Nivel2:

Funcionamiento Logrado y Limitaciones Actuales:

En el nivel isométrico se completó el movimiento lateral del barco, sistema de vidas con invulnerabilidad, munición limitada con recarga, torpedos destructores de obstáculos, sprint de velocidad, dificultad progresiva en 6 fases y una interfaz completa con temporizador.

Entre las limitaciones actuales destacan problemas de rendimiento por falta de optimización de renderizado y colisiones, control de volumen, un solo tipo de obstáculo sin variedad, y colisiones que no son precisas para formas circulares.

Dificultades Enfrentadas y Mejoras Planeadas

Las principales dificultades incluyen la coordinación de sistemas, sincronización entre proyección isométrica y colisiones 2D y colisiones múltiples durante la invulnerabilidad. Se resolvieron con conversiones de coordenadas y flags de invulnerabilidad con salida temprana.

Las mejoras planeadas priorizan completar el nivel top-down con colisiones y IA enemiga, implementar quadtree para optimizar colisiones, y añadir sistema de puntuación persistente. A mediano plazo se planea variedad de enemigos con comportamientos, sistema de partículas para explosiones, y audio mejorado con pool de sonidos. A largo plazo se considera migrar a OpenGL/Vulkan para mejor rendimiento e implementar algoritmo SAT para colisiones precisas con polígonos.

24- Conclusiones:

El desarrollo del videojuego permitió aplicar de manera práctica los conocimientos adquiridos durante el semestre, demostrando que es posible construir una experiencia interactiva completa utilizando Qt y C++. La estructura jerárquica del sistema facilitó la organización del código y el trabajo en equipo, logrando un producto funcional y escalable.

25- Reflexión sobre el proceso, el aprendizaje obtenido y recomendaciones

El proyecto fortaleció habilidades técnicas como el diseño de clases, manejo de colisiones y lógica de juego, así como competencias blandas como la comunicación y el trabajo colaborativo. Se recomienda para futuros proyectos una mejor planificación del tiempo, mayor optimización del rendimiento y la inclusión de más pruebas automatizadas.

26- Referencias

<https://www.youtube.com/watch?v=ZFHnbozz7b4>

<https://github.com/LightEievui/Zaxxon>

<https://github.com/Przemekkkth/top-down-game-sfml?tab=readme-ov-file>

<https://www.youtube.com/watch?v=pj3m3Fu3i5A>

27- Bibliografía, enlaces y documentación consultada:

<https://www.youtube.com/watch?v=fDDGn7e65pg>

<https://www.youtube.com/watch?v=OqwQBWEzcxU>

https://gamedev.stackexchange.com/questions/177422/proper-2d-isometric-tile-depth-sorting?utm_source=chatgpt.com

<https://www.youtube.com/watch?v=04oO2jOUjkU>

<https://www.youtube.com/watch?v=ukkbNKTgf5U>

<https://github.com/MeLikeyCode/QtGameEngine>

28-Manual de usuario (si aplica), capturas de pantalla, fragmentos de código importantes comentados, diagramas adicionales:

El jugador controla los personajes mediante teclado y mouse según el nivel seleccionado. El objetivo principal es cumplir las misiones de cada nivel evitando perder todas las vidas. El HUD muestra información relevante como vida, munición y progreso. Al finalizar cada nivel, se presenta una pantalla de victoria o derrota con opciones para reintentar o regresar al menú principal.

Nivel 2:

```
// Verificar colisiones entre torpedos y obstáculos
void NivelIso::verificarColisionesTorpedos()
{
    for (int i = m_torpedos.size() - 1; i >= 0; --i) {
        if (!m_torpedos[i].estaActivo()) {
            continue;
        }

        for (int j = m_obstaculos.size() - 1; j >= 0; --j) {
            bool colision = m_torpedos[i].hitbox().intersects(
                m_obstaculos[j].hitbox(),
                m_torpedos[i].position(),
                m_obstaculos[j].position()
            );

            if (colision) {
                if (m_sonidoExplosion) {
                    m_sonidoExplosion->play();
                }

                m_obstaculos.removeAt(j);
                m_torpedos[i].desactivar();
                break;
            }
        }
    }
}
```

```
* - El barco solo puede moverse en el eje Y (izquierda/derecha)
* - El eje X está reservado para el scrolling automático
*
* Convención de ejes:
*   x (profundidad)
*   ^
*  /
* /
* /
* -----> y (ancho)
*/
```

```

/*
 * paintEvent construye cada fotograma del nivel:
 * - pinta fondo con efecto de scrolling
 * - proyecta y dibuja el área jugable
 * - proyecta y dibuja barco y obstáculos
 * - dibuja las hitboxes de depuración
 */
void NivelIso::paintEvent(QPaintEvent *event)
{
    Q_UNUSED(event);

    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing, true);

    // Dibujar fondo con scrolling
    dibujarFondoScrolling(painter);

    /*
     * Transformación base:
     * - Colocamos el origen del sistema de coordenadas en el centro de la ventana.
     * - A partir de aquí, el mundo 2D se dibuja "alrededor" del centro.
     */
}

```